

SYSC 3303: Real Time Concurrent Systems

Elevator System Project - Final Report

Group B2-G5 :

JaniceTagak, 101172920
Nethul Bodiratne, 101277516
Uchenna Obikwelu, 101241887
Divya Dushyanthan, 101221637
Deborah Ajayi, 101220844

Date: 04/09/2025

Table of Contents

1. Introduction	3
2. Breakdown of Responsibilities	4
Iteration 1	4
Iteration 2	4
Iteration 3	4
Iteration 4	4
Iteration 5	4
3. Diagrams	5
Figure 1: UML Class Diagram	5
Figure 2: Elevator State Machine Diagram	6
Figure 3: Scheduler State Machine Diagram	6
Figure 4: Sequence Diagram	7
Figure 5: Timing Diagram	8
4. Setup and Test Instructions	8
System Execution Instructions (Multi-Terminal Setup)	8
Unit Test Instructions	8
Integration Test Instructions	9
5. Results from Measurements	9
6. Reflection on Design	11

1. Introduction

The development of our elevator scheduling system is structured into five key iterations, each progressively enhancing the functionality, efficiency, and robustness of the system. Our project centers around designing a multi-elevator control system that optimizes elevator movement, minimizes wait times, and gracefully handles faults. Through the use of multi-threading, inter-process communication, and networked components, we built a reliable and scalable solution.

Iteration 1: Establishing Communication

The foundation of our system is built in the first iteration, where we establish communication between the Floor, Scheduler, and Elevator subsystems. We implement basic message passing using multithreading, ensuring data flows seamlessly between components. At this stage, the Scheduler acted as a simple relay, forwarding requests between the Floor and Elevator without making scheduling decisions. Our focus here was on validating system connectivity and data integrity.

Iteration 2: Implementing the Scheduler and Elevator State Machines

With the communication layer in place, we introduced state machines for the Scheduler and Elevator subsystems. The Scheduler transitions from a passive relay to an active controller, managing a single elevator and tracking its state. The Elevator subsystem now responds dynamically to movement commands and signals the Scheduler upon reaching its destination. This iteration laid the groundwork for advanced scheduling logic in subsequent phases.

Iteration 3: Multi-Elevator Coordination and System Distribution

At this stage, we expanded the system to support multiple elevators operating independently. The system is refactored to run as three separate networked programs, communicating via UDP. The Scheduler now makes intelligent dispatching decisions, ensuring load balancing across elevators to optimize efficiency. We introduce algorithms that assign requests based on factors such as distance, direction, and current load.

Iteration 4: Fault Detection and Error Handling

To enhance reliability, we implemented fault detection mechanisms that monitor elevator movement and door operations. If an elevator fails to reach a floor within a specified time, the system assumes a failure and either retries the operation or shuts down the affected unit. We introduce test scenarios to simulate real-world failures such as stalled elevators and stuck doors, ensuring our system can recover gracefully from these faults.

Iteration 5: User Interface and Performance Optimization

In the final phase, we developed a real-time display system in the Scheduler console, that provides a visual representation of elevator positions and fault statuses. The system is further optimized to minimize travel time and improve throughput. Capacity limits are enforced, ensuring elevators do not exceed their designed load. We also incorporate performance tracking to measure system efficiency, culminating in a competitive challenge to determine the most optimized elevator scheduling algorithm.

Through this structured development process, we have designed a scalable and intelligent elevator scheduling system capable of handling real-world constraints. The final implementation showcases our team's expertise in concurrent programming, distributed systems, and fault-tolerant design.

2. Breakdown of Responsibilities

Iteration 1

- README: Nethul
- Diagrams: Janice
- Elevator Subsystem: Divya
- Scheduler: Uchenna
- Floor Subsystem: Deborah

Iteration 2

- Bug Fixes (Stop Issue & Others): Divya, Deborah, Uchenna
- New Features Implementation: Nethul, Janice
- README: Deborah
- UML Diagrams: Uchenna
- State Diagrams: Divya

Iteration 3

- Elevator: Nethul, Janice
- Floor: Deborah, Janice
- Scheduler: Uchenna, Divya
- Unit Test: Divya
- Convert Input File to Class: Janice
- README: Deborah
- Diagrams Review: Uchenna

Iteration 4

- Scheduler: Divya
- Elevator: Janice
- Floor: Deborah
- Tests: Uchenna
- State diagram: Divya, Janice
- Timing Diagram: Nethul
- UML Class Diagram: Deborah
- README: Deborah

Iteration 5

- Add Capacity Limits: Uchenna
- Display console: Divya
- State diagrams: Janice
- Timing Diagram: Nethul
- Sequence Diagram: Uchenna
- Unit Test: Uchenna
- Acceptance Test: Uchenna

- Integration Test: Nethul
- UML Class Diagram: Deborah
- README: Deborah

3. Diagrams

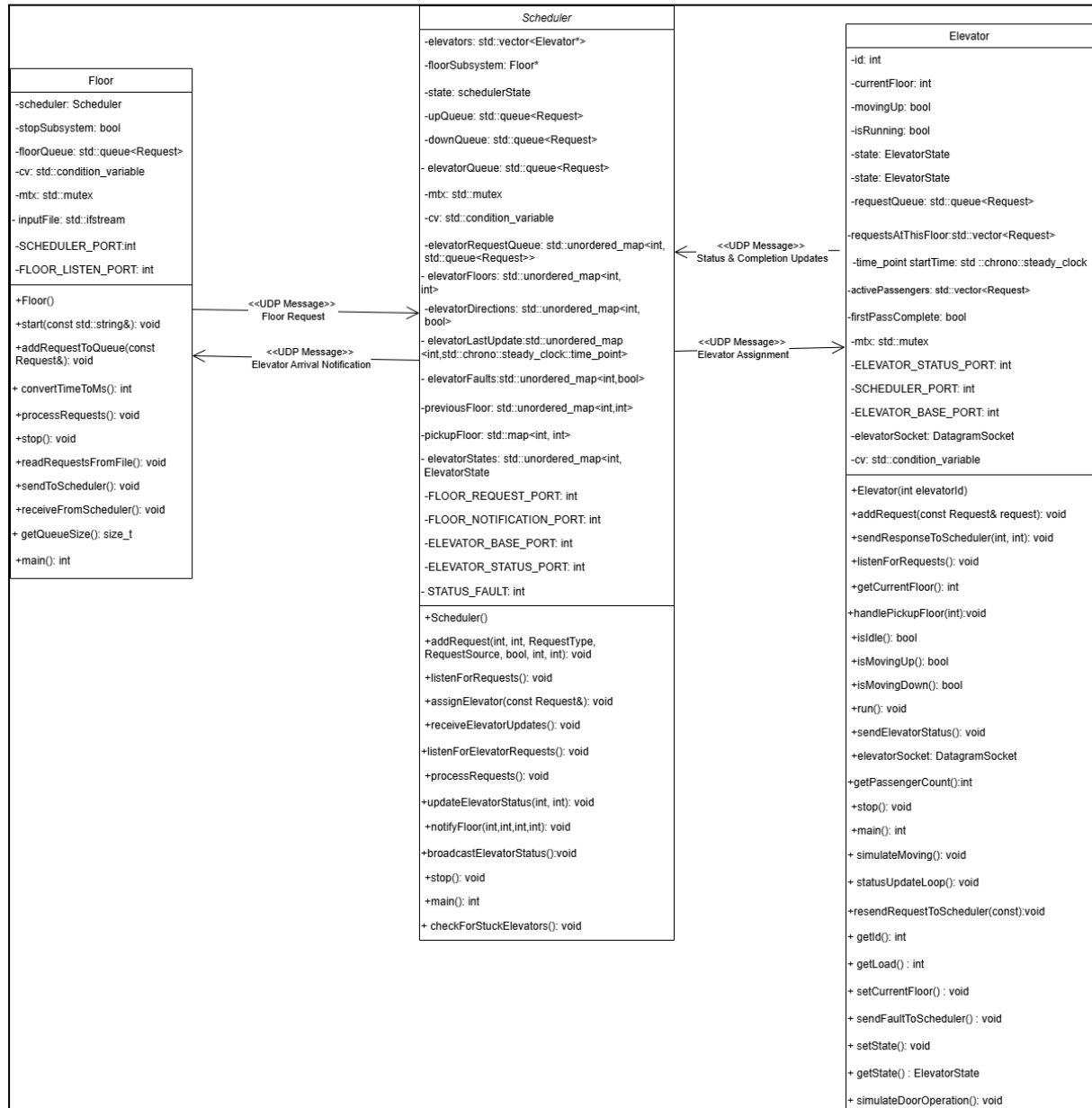


Figure 1: UML Class Diagram

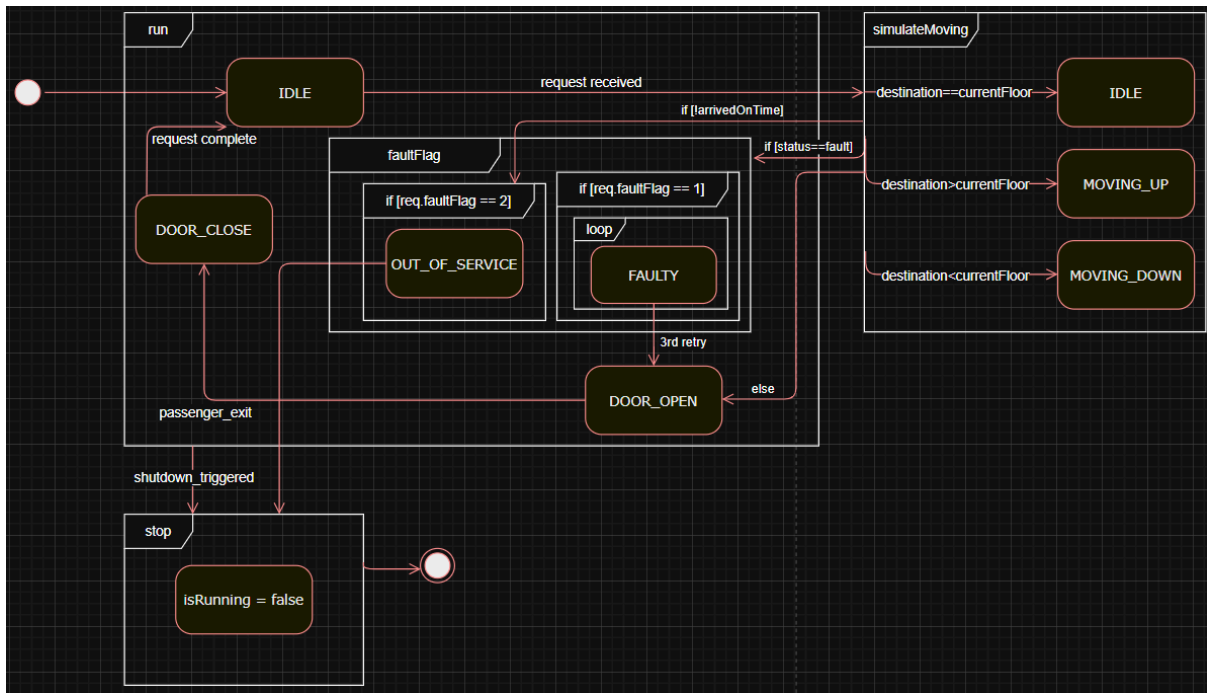


Figure 2: Elevator State Machine Diagram

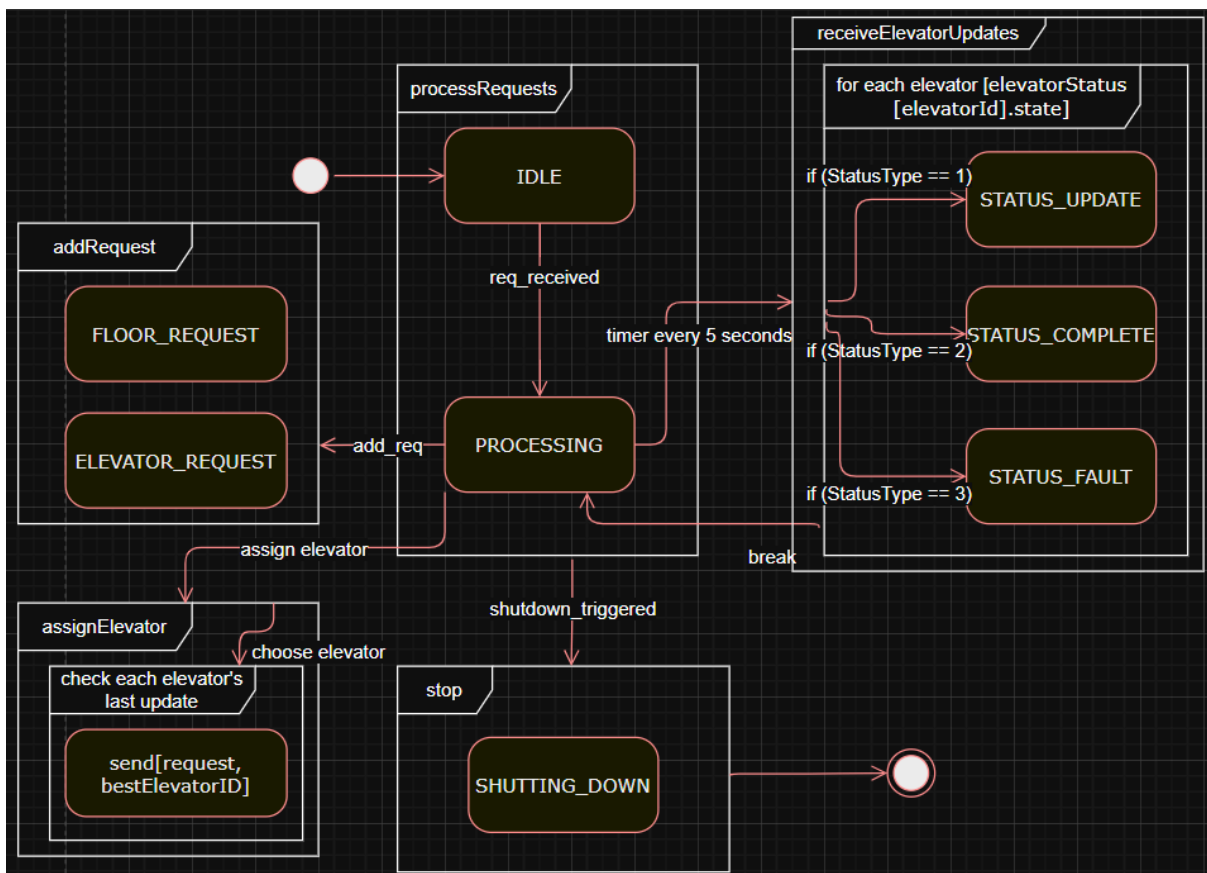


Figure 3: Scheduler State Machine Diagram

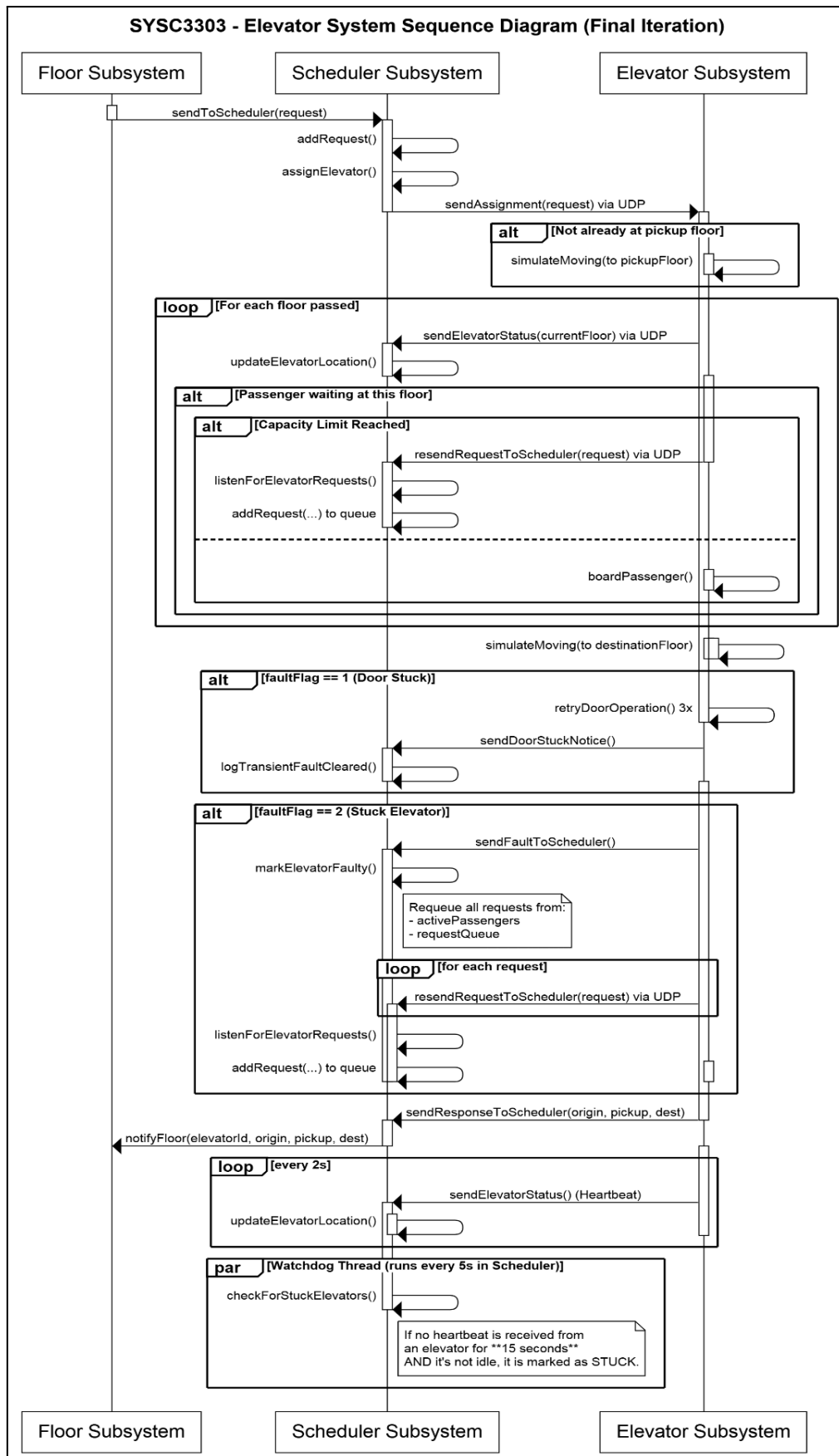


Figure 4: Sequence Diagram

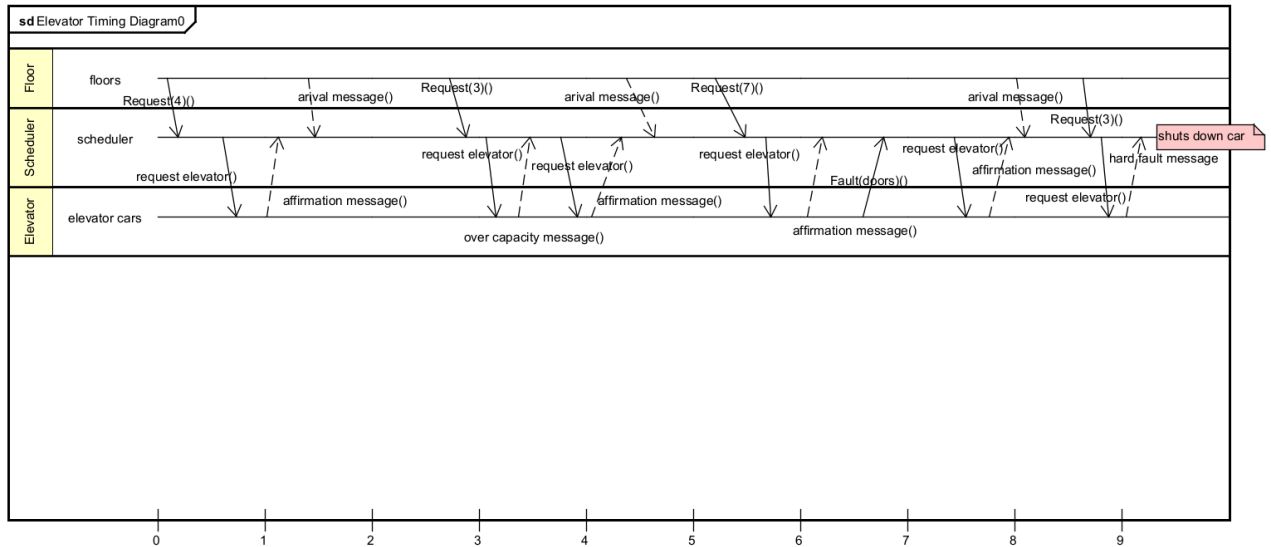


Figure 5: Timing Diagram

4. Setup and Test Instructions

System Execution Instructions (Multi-Terminal Setup)

You will use three terminals to run:

- 1) In Terminal 1, first compile the three subsystem files:

```
g++ -std=c++11 -o scheduler Scheduler.cpp
g++ -std=c++11 -o elevator Elevator.cpp
g++ -std=c++11 -o floor Floor.cpp
```

Then run:

```
./scheduler
```

- 2) In Terminal 2, run:

```
./elevator
```

- 3) In Terminal 3, run:

```
./floor
```

Unit Test Instructions

- 1) Compile with test flags:

```
g++ -DTEST_MODE -o elevator_tests unit_test.cpp Elevator.cpp Scheduler.cpp Floor.cpp -pthread
-std=c++14 -lgtest -lgtest_main
```


2) Run the tests:

```
./elevator_tests
```

Integration Test Instructions

1) Compile with test flags:

```
g++ -DTEST_MODE -o integration_tests IntegrationTest.cpp Elevator.cpp Scheduler.cpp Floor.cpp  
-pthread -std=c++14 -lgtest -lgtest_main
```

2) Run the tests:

```
./integration_tests
```

5. Results from Measurements

Trial	Requests	Movements	Time (s)
1	5	39	180
2	5	39	180
3	5	39	180
4	5	39	180
5	5	39	180

Calculate the Mean Time (μ):

The mean time (μ) is calculated as: $\mu = \sum X_i / n$

Mean Time(Average across all rounds)

$$\sum X_i = 180 + 180 + 180 + 180 + 180 = 900$$
$$n = 5$$
$$\mu = 900/5 = 180s$$

Mean Time per Movement = $180/39 = 4.615$ s

Where the Instrumentation was added

In this iteration, we instrumented the Elevator class to track and analyze its operational performance, specifically focusing on:

1. Total Movement Tracking

Variable Used: totalMovements (incremented every time the elevator moves one floor).

Where: Inside simulateMoving() loop after each floor is passed.

Purpose: Quantify total floor-to-floor transitions for each elevator.

2. Time Measurement

- Start Time: Tracked using:

`startTime = std::chrono::steady_clock::now()` during elevator initialization.

- End Time: Captured at the end of servicing a request via:

`std::chrono::steady_clock::now()` in `sendResponseToScheduler()`.

- Elapsed Time Calculation:

`duration = std::chrono::duration_cast<std::chrono::seconds>(endTime - startTime).count();`

Purpose: Report how long an elevator has been running since it started.

What the Value Means:

This value represents the cumulative operational time (in seconds) that the elevator has been running. It helps contextualize performance metrics, such as requests handled per second or total requests serviced during the simulation.

Mean time per event, derived or measured

In our case, the mean time per event was derived, not directly measured for every individual event. Specifically, the method used was:

Events were timed as a group using a high-resolution timer. The total time was then divided by the number of events to obtain an average or "mean time per event."

This approach was taken because the timer resolution available on the system was not sufficient to measure each event accurately on its own — many individual events completed faster than the timer's smallest measurable interval (insufficient clock resolution). Averaging over a large number of events reduces the effect of measurement error and gives a more reliable estimate of execution time per event.

Calculate 95% Confidence Interval (CI)

$CI = \mu \pm t \times (\sigma / \sqrt{n}) = 180 \pm t \times 0 = 180$

This 95% Confidence Interval: [180, 180]seconds

So the mean is highly stable and reliable estimate

Analysis of time estimates

The analysis was conducted using controlled test inputs and measured by extracting real-time performance metrics from the program's execution. The following steps were taken:

Step 1: Code Modifications for Performance Tracking

1. The elevator code was modified to track and display:

- The total number of floor movements (totalMovements)
- The total time taken (duration) from the start of the elevator's operation to request completion

This information was printed in the console output after each request was processed.

Step 2: Test Input Execution

A set of predefined test inputs was used to evaluate the elevator's performance. Some test cases consisted of multiple requests sent to the elevator system, including different pickup and destination floors.

Step 3: Multiple Runs for Consistency

To ensure accuracy, some input sets were executed three times, and performance metrics were recorded. This helped analyze variations in response time due to different scheduling scenarios.

Step 4: Data Collection

The key metrics (totalMovements and totalTime) were collected from the console output for each test

The code was instrumented using time markers around the movement execution loop to measure total elapsed time for each round. Five runs were conducted using the same input (5 requests), and each round required 39 movements, taking exactly 180 seconds. As all measurements were identical, the mean execution time per round was 180s, and the mean time per movement was approximately 4.615s. The standard deviation was 0, resulting in a confidence interval of [180, 180] seconds at 95% confidence, confirming the consistency of the measurement. Due to limitations in timing individual events precisely, total time was used and then divided by the number of movements to obtain the mean time per event.

6. Reflection on Design

In the Elevator Scheduling System project, we successfully integrated capacity constraints, fault detection, and a real-time monitoring interface, culminating in a fully functional and realistic simulation of an elevator system. We implemented logic to prevent elevator overloading, requeue passengers, and handle faults such as stuck doors and elevator stalls through timed event triggers and system responses.

Our team collaborated effectively by dividing tasks equally and maintaining consistent communication through frequent in-person and online meetings, especially during debugging and system integration.

In hindsight, we could have benefited from a more easily scalable Scheduler and Elevator implementation design in the earlier iterations. Additional improvements could include better documentation, simulating variable passenger behavior more realistically, and integrating a more user-friendly front-end display.

Our favourite parts of the design include: integrating clean and correct UDP communication between subsystems, the detailed and informative consoles for each subsystem, the robustness of our test cases, and how easy the code was to trace during debugging. These test cases and clear code structure allowed us to identify defects early and validate system behavior thoroughly.

Overall, this final iteration highlighted our team's growth in system design, concurrency handling, and collaborative problem-solving.